

DIGILIANS Initiative

Capstone Project

Full Stack Development Track

Sa2yanti



Car Maintenance Services Platform

Prepared By:

Rabea Shaban

Individual Project

Submitted in Partial Fulfillment of the Requirements for the
Capstone Project – Full Stack Development Track

DIGILIANS Initiative

2026

وزارة الاتصالات
وتكنولوجيا المعلومات



Page	Content
1	Cover Page
2	Abstract
3-4	Introduction
5-6	Problem Definition
7	Project Objectives
8	System Architecture
9	Frontend Architecture
10	Backend Architecture
11	Database Design
12	Implementation & Dependencies
13	Achievements & Progress
14	Challenges & Future Work
15	References

Abstract

Sa2yanti is a web-based platform designed to connect vehicle owners with maintenance technicians through a centralized digital system. The platform aims to simplify the process of requesting maintenance services, tracking service progress, and improving communication between customers and service providers.

The system is developed using modern Full Stack technologies, including React.js and TypeScript for the frontend, Node.js and Express.js for the backend, and MongoDB Atlas for database management. To ensure secure access and user data protection, the platform implements JWT authentication with HTTP Only Cookies and Role-Based Access Control.

The current implementation focuses on core functionalities such as user authentication, authorization, and maintenance request management. Users can register, log in securely, and create maintenance requests through an organized and user-friendly interface.

The platform architecture is designed to be scalable and maintainable, allowing future enhancements such as technician dashboards, notifications, service tracking, and location-based features.

Sa2yanti aims to improve the overall vehicle maintenance experience by providing a secure, efficient, and centralized solution that benefits both vehicle owners and maintenance service providers.

1. Introduction

The automotive industry is an essential part of daily life, and the growing number of vehicles has increased the demand for maintenance services. However, many vehicle owners still face challenges in finding reliable technicians and managing maintenance requests efficiently.

Traditional maintenance processes often rely on phone calls, personal recommendations, or social media communication, which can be time-consuming and difficult to manage. In addition, customers usually have limited visibility into the status of their service requests.

To address these challenges, Sa2yanti was developed as a web-based platform that connects vehicle owners with maintenance technicians through a centralized digital system. The platform allows users to create maintenance requests, track their progress, and access services in an organized manner.

The project is built using modern Full Stack technologies, including React.js, TypeScript, Node.js, Express.js, and MongoDB Atlas. Secure authentication is implemented using JWT and HTTP Only Cookies.

Sa2yanti aims to improve communication between customers and service providers while providing a secure, efficient, and scalable solution for vehicle maintenance management.

2. Problem Definition

The automotive maintenance sector still relies heavily on traditional communication methods, which often lead to inefficiencies and poor service management. Vehicle owners frequently face difficulties in finding reliable technicians, tracking maintenance requests, and receiving timely updates about service progress.

In many cases, maintenance requests are handled through phone calls or messaging applications, making it difficult to organize requests, monitor service status, and maintain accurate records. This lack of structure can result in delays, misunderstandings, and reduced customer satisfaction.

From the service provider's perspective, managing requests manually becomes increasingly challenging as the number of customers grows. In addition, many existing solutions lack proper security mechanisms and role-based access control, which may affect data protection and system reliability.

Therefore, there is a need for a centralized digital platform that simplifies maintenance request management, improves communication between customers and technicians, and provides secure authentication and authorization.

Sa2yanti addresses these challenges by offering a web-based platform where users can create and track maintenance requests, while technicians can efficiently manage and update service operations through a secure and organized system.

3. Project Objectives

The main objective of the Sa2yanti project is to develop a web-based platform that simplifies the process of requesting and managing vehicle maintenance services. The platform aims to connect vehicle owners with maintenance technicians through a secure and organized digital environment.

The project focuses on achieving the following objectives:

- Provide a centralized platform for maintenance service requests.
- Improve accessibility to maintenance services.
- Implement secure authentication and authorization mechanisms.
- Enable efficient order creation and management.
- Enhance user experience through a responsive interface.
- Develop a scalable Full Stack architecture.
- Apply modern software development technologies and practices.
- Support future enhancements and feature expansion.

Expected Outcomes

Upon completion, the platform is expected to:

- Improve communication between vehicle owners and technicians.
- Simplify maintenance request management.
- Provide a secure and reliable service environment.
- Increase transparency throughout the maintenance process.
- Establish a scalable foundation for future development.

By achieving these objectives, Sa2yanti aims to provide an effective digital solution that improves the overall vehicle maintenance experience for both customers and service providers.

4. System Architecture

Sa2yanti follows a three-tier architecture consisting of the frontend layer, backend layer, and database layer. This architecture improves scalability, maintainability, and security while ensuring efficient communication between system components.

Architecture Overview

User

↓

Frontend (React.js)

↓

REST API (Node.js & Express.js)

↓

Database (MongoDB Atlas)

Frontend Layer

The frontend is developed using React.js and TypeScript and is responsible for user interaction and interface management.

Main technologies:

- React.js
- TypeScript
- Tailwind CSS
- React Router DOM
- Axios
- Context API

Main responsibilities:

- Displaying user interfaces.
- Managing navigation and routing.
- Handling forms and validation.
- Communicating with backend APIs.

Backend Layer

The backend is developed using Node.js and Express.js and contains the application's business logic.

Main technologies:

- Node.js
- Express.js
- JWT
- HTTP Only Cookies
- Joi Validation
- bcryptjs

Main responsibilities:

- Authentication and authorization.
- Request validation.
- Order management.
- API development.
- Database communication.

Database Layer

MongoDB Atlas is used as the primary database for storing application data.

Stored data includes:

- User information.
- Maintenance requests.
- Technician assignments.
- Service status records.

Technologies:

- MongoDB Atlas
- Mongoose ODM

Authentication Flow

1. User logs in using valid credentials.
2. The server generates a JWT token.
3. The token is stored in an HTTP Only Cookie.
4. Protected routes verify the token through middleware.

5. Authorized users gain access to system resources.

Benefits

- Clear separation of responsibilities.
- Secure authentication and authorization.
- Easy frontend-backend integration.
- Scalable and maintainable architecture.
- Support for future enhancements and additional features.

5. Frontend Architecture

The frontend of Sa2yanti is responsible for providing a responsive and user-friendly interface for vehicle owners and technicians. It enables users to interact with the platform, manage maintenance requests, and access system features efficiently.

The application is developed using React.js and TypeScript following a component-based architecture that improves maintainability and code reusability.

Technologies Used

The frontend is built using:

- React.js
- TypeScript
- Tailwind CSS
- React Router DOM
- React Hook Form
- Zod
- Axios
- Context API

Frontend Structure

The project follows a modular structure consisting of:

- Components
- Pages
- Context
- Hooks
- Routes
- Services

This structure improves code organization and simplifies future development.

Authentication Flow

Authentication is managed using Context API and HTTP Only Cookies.

Process:

1. User submits login credentials.
2. Backend validates the user and generates a JWT token.
3. The token is stored in an HTTP Only Cookie.
4. User information is retrieved and stored in Context API.
5. Protected routes become accessible after successful authentication.

Implemented Pages

- Landing Page
- Login Page
- Register Page
- Protected Pages

Benefits

- Responsive user interface.
- Reusable components.
- Secure authentication integration.
- Easy API communication.
- Scalable architecture for future enhancements.

The frontend architecture provides a solid foundation for future modules such as order tracking, technician dashboards, and additional maintenance services.

6. Backend Architecture

The backend of Sa2yanti is responsible for handling business logic, authentication, authorization, and database communication. It is developed using Node.js and Express.js following a RESTful API architecture.

Technologies Used

The backend is built using:

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- JWT
- HTTP Only Cookies
- Joi Validation
- Bcryptjs

Backend Structure

The project follows a modular architecture consisting of:

- Models
- Controllers
- Routes
- Middleware
- Validators
- Config

This structure improves code organization, maintainability, and scalability.

Authentication and Authorization

Authentication is implemented using JWT and HTTP Only Cookies.

Process:

1. User submits login credentials.
2. Credentials are validated.
3. JWT token is generated.
4. Token is stored in an HTTP Only Cookie.
5. Middleware verifies the token before granting access.

Role-Based Access Control (RBAC) is used to manage permissions for different user roles such as users and technicians.

REST API

The backend provides RESTful APIs for:

Authentication

- Register User
- Login User
- Logout User
- Get Current User

Orders

- Create Order
- View User Orders

Security Features

The backend includes several security mechanisms:

- Password hashing using bcryptjs.
- JWT authentication.
- HTTP Only Cookies.
- Request validation using Joi.
- Protected API routes.
- Role-based authorization.

Benefits

- Secure authentication and authorization.
- Organized and maintainable code structure.
- Efficient database communication.
- Easy integration with the frontend.
- Scalable architecture for future development.

The backend architecture provides a reliable foundation for managing maintenance requests and supporting future platform features.

7. Database Design

The Sa2yanti platform uses MongoDB Atlas as its primary database for storing user information and maintenance requests. The database is designed to provide scalability, flexibility, and efficient data management.

Technologies Used

- MongoDB Atlas
- Mongoose ODM

MongoDB stores data in collections and documents, while Mongoose is used to define schemas and manage database operations.

User Collection

The User collection stores information about platform users.

Main fields:

- Name
- Email
- Phone
- Password
- Role

Supported roles:

- User
- Technician

Order Collection

The Order collection stores maintenance requests submitted by users.

Main fields:

- User ID
- Technician ID
- Service
- Location
- Status

Order statuses include:

- Pending
- Accepted
- Completed

Relationships

The platform uses the following relationships:

- One User can create multiple Orders.
- One Technician can manage multiple Orders.

This structure ensures proper management and tracking of maintenance requests.

Data Validation and Security

To maintain data integrity and security:

- Joi is used for backend validation.
- Passwords are hashed using bcryptjs.
- Authentication and authorization middleware protect database operations.
- User roles are validated before accessing protected resources.

Benefits

- Flexible and scalable database structure.
- Efficient data storage and retrieval.
- Easy integration with Node.js applications.
- Support for future platform expansion.

The database design provides a reliable foundation for managing users, technicians, and maintenance requests within the Sa2yanti platform.

8. Implementation & Dependencies

The Sa2yanti platform was developed using a Full Stack architecture consisting of a React.js frontend, Node.js backend, and MongoDB Atlas database.

Implemented Features

- User Registration and Login.
- JWT Authentication with HTTP Only Cookies.
- Role-Based Authorization.
- Create Order Functionality.
- Protected Routes.
- Request Validation.
- Database Integration.

Technologies Used

Frontend

- React.js
- TypeScript
- Tailwind CSS
- React Router DOM
- Axios
- React Hook Form
- Zod

Backend

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- JWT
- bcryptjs
- Joi
- cookie-parser

Benefits

- Secure authentication.
- Responsive user interface.
- Scalable architecture.
- Clean and maintainable code structure.

The current implementation provides the core functionality of the platform and serves as a foundation for future enhancements.

9. Achievements & Progress

The Sa2yanti project has successfully completed the foundation phase of development. Several core features have been implemented and integrated successfully.

Completed Features

- User Registration and Login.
- JWT Authentication with HTTP Only Cookies.
- Role-Based Authorization (User & Technician).
- User and Order Database Models.
- Create Order API.
- Protected Routes.
- Request Validation using Joi and Zod.
- Landing Page, Login Page, and Register Page.
- Frontend and Backend Integration.

Testing

The implemented features were tested using Postman and browser testing, including:

- User Authentication.
- Authorization Validation.
- Order Creation.
- Protected Route Access.

Current Status

Completed

- Authentication System.
- Authorization System.
- Database Integration.
- Orders Backend Module.
- API Integration.
- Frontend Authentication Pages.

In Progress

- Orders Frontend Module.
- My Orders Page.
- User Dashboard.

Future Work

- Technician Dashboard.
- Order Status Management.
- Notifications System.
- Maps Integration.
- Reviews and Ratings.

The project currently provides a stable foundation and is progressing according to the planned development timeline.

10. Challenges & Future Work

During the development of Sa2yanti, several challenges were encountered, particularly in authentication, authorization, and frontend-backend integration. One of the main challenges was migrating from Local Storage authentication to JWT authentication using HTTP Only Cookies, which required modifications to both frontend and backend workflows.

Another challenge was implementing Role-Based Access Control (RBAC) to ensure that users and technicians could only access authorized resources. These challenges were addressed through authentication middleware, role middleware, and secure API design.

Future Work

The current version provides the core functionality of the platform. Future enhancements include:

- My Orders Page.
- Technician Dashboard.
- Order Status Management.
- Notifications System.
- Location-Based Services.
- Reviews and Ratings.
- Advanced Analytics and Reporting.

These features will improve user experience and expand the platform into a complete vehicle maintenance management solution.

Summary

The project has successfully established a secure and scalable foundation for managing maintenance requests. Future development will focus on adding advanced features and enhancing the overall service experience.

11. References

The following resources were used during the development of the Sa2yanti platform:

Frontend

- React.js Documentation
- TypeScript Documentation
- Tailwind CSS Documentation
- React Router DOM Documentation
- React Hook Form Documentation
- Zod Documentation
- Axios Documentation

Backend

- Node.js Documentation
- Express.js Documentation
- MongoDB Atlas Documentation
- Mongoose Documentation
- JSON Web Token (JWT) Documentation
- Joi Documentation
- bcryptjs Documentation

Development Tools

- Visual Studio Code
- Postman
- Git & GitHub

Academic References

- Pressman, R. S. *Software Engineering: A Practitioner's Approach*.
- Sommerville, I. *Software Engineering*.
- Elmasri, R., & Navathe, S. *Fundamentals of Database Systems*.

These references provided the technical and theoretical foundation for the design and implementation of the Sa2yanti platform.